COURSE PACK

SCHEME

The scheme is an overview of work-integrated learning opportunities and gets students out into the real world. This will give what a course entails.

Course Title	**Design & Analysis of Algorithms**			**Course Type**				**Comprehensive**	
Course Code	R1UC407B			**Class**				B.Tech Core and All specialization	
Instruction delivery	Activity	Credits	Credit Hours	Total Number of Classes per Semester				Assessment in Weightage	
	Lecture	3	4	Theory	Tutorial	Practical	Self-Learning	CIE	SEE
	Tutorial	0	0						
	Practical	2	4						
	Self-Learning	0	0						
	Total	5	8	60	0	60	0	**50%**	**50%**

- **Names Course Coordinators and Instructors**

- **Course Lead:**
- **Coordinator:**

- Dr. Ashok Kumar Yadav
- Dr. Mili Dhar

Theory	Practical
Abhishek Kumar Pandey	Abhishek Kumar Pandey
Adarsh Kumar	Adarsh Kumar
Akhilesh Kumar Tripathi	Akhilesh Kumar Tripathi
Albert Mundu	Albert Mundu
Ambika Gupta	Ambika Gupta
Ankit Sharma	Ankit Sharma
Ankit Verma	Ankit Verma
Anurag maurya	Anurag maurya
Arpan Kumari	Arpan Kumari
G. Sakthi	G. Sakthi
Jyoti Yaduwanshi	Jyoti Yaduwanshi
Mandeep	Mandeep
Mili Dhar	Mili Dhar
Mithilesh Kumar Yadav	Mithilesh Kumar Yadav
N. Partheeban	N. Partheeban
Neha Singh	Neha Singh
Pradeep Chauhan	Pradeep Chauhan
Radha Rani	Radha Rani
Rakesh Bharati	Rakesh Bharati
Sachin Jain	Sachin Jain

	• Sachin Kumar Tyagi • Shachi Mall • Sunil Kumar • Tushar Mehrotra • Vartika Mishra • Sunil Kumar		• Sachin Kumar Tyagi • Shachi Mall • Sunil Kumar • Tushar Mehrotra • Vartika Mishra • Sunil Kumar

COURSE OVERVIEW

An Algorithm is a sequence of steps to solve a problem. Design and Analysis of Algorithm is very important for designing algorithm to solve different types of problems in the branch of computer science and information technology. This tutorial introduces the fundamental concepts of Designing Strategies, Complexity analysis of Algorithms, followed by problems on Graph Theory and Sorting methods. This tutorial also includes the basic concepts on Complexity theory.

PREREQUISITE COURSE

PREREQUISITE COURSE REQUIRED	YES	

If, yes please fill in the details

Prerequisite course code	Prerequisite course name
R1UC303B	Data Structures using Java

COURSE OBJECTIVES

- Analyze and design different searching and sorting algorithm algorithms based upon different designing approaches.
- Analyze and design different tree algorithms based upon different designing approaches.
- Design efficient algorithmic techniques for solving industry oriented real time problems using Greedy Algorithm.
- Design efficient algorithmic techniques for solving industry oriented real time problems using Dynamic Programming.
- Apply and synthesize efficient algorithms.

COURSE OUTCOMES(COs)

After the completion of the course, the student will be able to

C01	Design new algorithms, prove them correct, and analyze their asymptotic and absolute runtime and memory demands.
C02	Find an algorithm to solve the problem (create) and prove that the algorithm solves the problem correctly (validate).

C03	Understand the mathematical criterion for deciding whether an algorithm is efficient, and know many practically important problems that do not admit any efficient algorithms.
C04	Apply classical sorting, searching, optimization and graph algorithms.
C05	Understand basic techniques for designing algorithms, including the techniques of recursion, divide-and-conquer, and greedy.

BLOOM'S LEVEL OF THE COURSE OUTCOMES

Bloom's taxonomy is a set of hierarchical models used for the classification of educational learning objectives into levels of complexity and specificity. The learning domains are cognitive, affective, and psychomotor**.**

COMPREHENSIVE

CO No.	Remember **BTL1**	Understand **BTL2**	Apply **BTL3**	Analyse **BTL4**	Evaluate **BTL5**	Create **BTL6**
C01				√		
C02						√
C03					√	
C04				√		
C05			√			

PROGRAM OUTCOMES (POs): **PO1 Computing Science knowledge:** Apply the knowledge of mathematics, statistics, computing science and information science fundamentals to the solution of complex computer application problems.

PO2 Problem analysis: Identify, formulate, review research literature, and analyze complex computing science problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and computer sciences**.**

PO3 Design/development of solutions: Design solutions for complex computing problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4 Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5 Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern computing science and IT tools including prediction and modeling to complex computing activities with an understanding of the limitations.

PO6 IT specialist and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional computing science and information science practice.

PO7 Environment and sustainability: Understand the impact of the professional computing science solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO8 Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the computing science practice.

PO9 Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO10 Communication: Communicate effectively on complex engineering activities with the IT analyst community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11 Project management and finance: Demonstrate knowledge and understanding of the computing science and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12 Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Program Specific Outcomes (PSO's)

PSO1: Have the ability to work with emerging technologies in computing requisite to Industry 4.0.

PSO2: Demonstrate Engineering Practice learned through industry internship and research project to solve live problems in various domains.

COURSE ARTICULATIONMATRIX

The Course articulation matrix indicates the correlation between Course Outcomes and Program Outcomes and their expected strength of mapping in three levels (low, medium, and high).

COs#/ POs	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2
201C.1	3	3	3										2	
201C.2	3	2	2		2								2	
201C.3	3	2	2										2	
201C.4	3	2	2										2	
201C.5	3	3	3	3	3			3	3				2	

Note: 1-Low, 2-Medium, 3-High

COURSE ASSESSMENT

The course assessment patterns are the assessment tools used both in formative and summative examinations.

Type of Course (B)	CIE			Total Marks		Final Marks
	LAB Work@ + Record	MTE	LAB EXAM*	CIE	SEE	CIE*0.5+SEE*0.5
INTEGRATED	25	50	25	100	100	100

@Lab Work-15 marks + Lab Record-10 marks

*Passing Criteria-30% of marks to be secured in the lab Exam conducted by two examiners (one internal and one external)

COURSE CONTENT

THEORY

Content
Introduction Notion of an Algorithm, Fundamentals of Algorithmic Problem Solving, Problem Types, Analysis of Algorithm Efficiency, Asymptotic Notations and their properties, Analysis Framework, Empirical Analysis, Mathematical analysis for Recursive and Non-recursive algorithms, Solving Recurrence relations. **Brute Force and Divide-and-Conquer** **Brute Force**: String Matching, Traveling Salesman Problem, Knapsack Problem, Assignment problem. **Divide and conquer**: Merge sort, Quick sort, Binary search, Heap sort, Strassen's Matrix Multiplication **Greedy Technique and Dynamic Programming** **Greedy Technique**: Minimum spanning tree, Prim's and Kruskal's Algorithm, Huffman Trees and Codes, fractional Knapsack Problem, Graph algorithms: Topological sort, Dijkstra's Algorithm. **Dynamic Programming**: Principle of optimality. Coin-Exchange Problem, Longest Common Subsequence Problem, Travelling Salesman Problem, Multi stage graph, All pair shortest paths: Floyd–Warshall algorithm, Knapsack Problem. **Backtracking and Branch & Bound** **Backtracking**: n-Queens problem, Subset sum problem, Graph Coloring **Branch & Bound**: Knapsack Problem, Traveling Salesman Problem **Iterative Improvement and Limitations of Algorithm Power:** Introduction to P, NP NP- Complete and NP Hard Problems, Introduction to Approximation algorithms, Approximation Algorithms for NP-Hard Problems, Traveling Salesman problem, Knapsack problem.

PRACTICAL

Contents	
1	Write a program to sort given set of numbers in ascending/descending order using quick sort and also search a number using binary search.
2	Write a program to sort given set of numbers in ascending/descending order using merge sort and also search a number using linear search.
3	Write a program to sort given set of numbers using Heap sort.
4	Write a program to implement String Matching using naive algorithm.
5	Write a program to implement String Matching using KMP algorithm.
6	Write a program to implement Knapsack using Greedy technique.
7	Write a program to implement Knapsack using Dynamic programming.
8	Write a program to implement Dijkstra’s Algorithm.
9	Write a program to implement Dijkstra’s Algorithm.
10	Write a program to implement prim's Algorithm.
11	Write a program to implement prim's Algorithm.
12	Write a program to implement kruskal's Algorithm.
13	Write a program to implement kruskal's Algorithm.
14	Write a program to implement TSP.
15	Write a program to implement TSP.
16	Write a program to implement n-Queen Problem using backtracking.
17	Write a program to implement n-Queen Problem using backtracking.
18	Write a program to implement Sum of subset problem.
19	Write a program to implement solve LCS problem.
20	Write a program to implement solve LCS problem.

LESSON PLAN FOR THEORY COURSES (15 weeks * 3 Hours = 45 Classes)

LNo	T/L	Topics	Skills	Competency
1	T	Notion of an Algorithm, Fundamentals of Algorithmic Problem Solving, Problem Types,	Designing Algorithms and checking the complexity	Designing algorithms, analyze their space and time complexities.
2	T	Analysis of Algorithm Efficiency, Asymptotic Notations and their properties		
3	T	Asymptotic Notations((., Ω, Θ) based on Orders of Growth, Asymptotic Notations numerical		
4	L	Analysis Framework, Empirical Analysis		
5	L	Analysis Framework, Empirical Analysis		
6	T	Complexity analysis of non-recursive/iterative algorithms	Analyze the order of the growth of any algorithm	
7	T	Complexity analysis of recursive algorithms: substitution method	Analyze the complexity of any algorithm with recurrence relation	
8	T	Complexity analysis of recursive algorithms: Recursion tree method		
9	L	Complexity analysis of recursive algorithms: Master Theorem		
10	L	String Matching: Naïve algorithm	Designing and using Brute Force methods	
11	T	String Matching: KMP algorithm		
12	T	Brute Force: Knapsack Problem		
13	T	Brute Force: Assignment problem		
14	L	Divide and Conquer: Merge sort, Quick sort	Searching and sorting Algorithms Designing and analysis of divide and conquer methods	
15	L	Divide and Conquer: Binary Search		Analyze and apply searching and sorting algorithms
16	T	Divide and Conquer: Strassen's Matrix Multiplication		Finding the optimal solution using different approaches and analyze the problem
17	T	Greedy Technique: Minimum spanning tree Prim's algorithm	Designing and using greedy methods	
18	T	Greedy Technique: Minimum spanning tree Prim's algorithm		

19	L	Greedy Technique: Minimum spanning tree Kruskal's algorithm	Designing and using greedy methods	
20	L	Greedy Technique: Minimum spanning tree Kruskal's algorithm		
21	T	Greedy Technique: Huffman Trees and Codes		
22	T	Greedy Technique: Huffman Trees and Codes		
23	T	Greedy Technique: Fractional Knapsack		
24	T	Greedy Technique: Fractional Knapsack		
25	L	Greedy Technique: Topological Sort		
26	L	Greedy Technique: Dijkstra's algorithm		
27	T	Greedy Technique: Dijkstra's algorithm		
28	T	Dynamic Technique: Principle of optimality.		
29	T	Dynamic Technique: 0/1 Knapsack Problem.		
30	T	Dynamic Technique: Floyd- Warshall		
31	L	Dynamic Technique: Longest Common Subsequence Problem		
32	L	Dynamic Technique: Longest Common Subsequence Problem		
33	T	Dynamic Technique: Travelling Salesman Problem		
34	T	Dynamic Technique: Multistage graph		
35	T	Dynamic Technique: Coin-Exchange Problem		
36	L	Dynamic Technique: Coin-Exchange Problem		
37	L	Backtracking: n-Queens problem		
38	T	Backtracking: Subset sum problem		
39	T	Backtracking: Graph Coloring		
40	T	Branch & Bound: Traveling Salesman Problem		
41	L	Introduction to P, NP NP- Complete and NP Hard Problems.		
42	L	Revision and doubt clearing session		
43	T	Revision and doubt clearing session		
44	T	Revision and doubt clearing session		
45	T	Revision and doubt clearing session		

T= Theory, L= Lab

BIBLIOGRAPHY

Text Book

1. Thomas H. Coreman, Charles E. Leiserson and Ronald L. Rivest, "Introduction to Algorithms", The MIT Press, 3rd edition, 2009.

Reference Books

1. Ellis Horowitz, Sartaj Sahni, Sanguthevar Rajasekaran. Fundamentals of Computer Algorithms, MIT Press, Second Edition (Indian reprint: Prentice-Hall), 2008.
2. Aho, Hopcraft, Ullman, "The Design and Analysis of Computer Algorithms" Pearson Education.

Journals/Magazines/Govt.Reports/Gazatte/IndustryTrends

1. https://leetcode.com

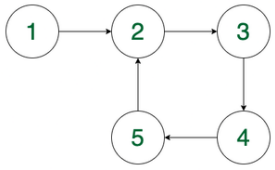
SWAYAM/NPTEL/MOOCs Certification

1. Code Tantra
2. Coding Ninjas
3. Code Chef
4. https://onlinecourses.nptel.ac.in/noc19_cs47/preview (Swayam)
5. GeeksforGeeks

PROBLEM-BASED LEARNING

Exercises in Problem-based Learning (Value Added Experiments)

SNo	Problem	KL
1	Write a program to implement Minimum Cost spanning tree.	KL5
2	Write a program to implement Sum of subset problem.	KL5
3	Write a program to implement Greedy algorithm using Task Scheduling Problem.	KL5
4	Write a program to implement Greedy algorithm using Activity Selection Problem.	KL5
5	Compute the transitive closure of a given directed graph using Warshall's algorithm. Write a program to implement shortest path algorithm.	KL6
6	Write a program to find all Hamiltonian Cycles in a connected undirected Graph G of n vertices using backtracking principle.	KL5
7	Develop a program to implement solve LCS problem.	KL6
8	Develop a program to implement Huffman-code.	KL6
9	Find Minimum Cost Spanning Tree of a given connected undirected graph using Kruskal's algorithm. Use Union-Find algorithms in your program.	KL6
10	Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.	KL6
11	Design a program to (a) Implement All-Pairs Shortest Paths problem using Floyd's algorithm. (b) Implement Travelling Sales Person problem using Dynamic programming.	KL6
12	Design and implement to find a subset of a given set S = {Sl, S2,.... , Sn} of n positive integers whose SUM is equal to a given positive integer d. For example, if S ={1, 2, 5, 6, 8} and d= 9, there are two solutions {1,2,6}and {1,8}. Display a suitable message, if the given problem instance doesn't have a solution.	KL6
13	Transpose of Matrix	KL5
14	Given an (n x m) matrix, rotate the matrix by 90 degrees in clockwise direction.	KL6
15	Given an array arr[] of size N-1 with integers in the range of [1, N], the task is to find the missing number from the first N integers.	KL6
16	Given a string, the task is to count all palindrome sub string in a given string. Length of palindrome sub string is greater than or equal to 2.	KL6
17	Given a set of strings, find the longest common prefix.	KL5
18	Given two strings, the task is to find if a string can be obtained by rotating another string by two places.	KL6
19	Given an expression string exp, write a program to examine whether the pairs and the orders of "{", "}", "(", ")", "[", "]" are correct in the given expression.	KL5
20	Given a string, reverse it using stack.	KL5

21	Given an integer k and a queue of integers, The task is to reverse the order of the first k elements of the queue, leaving the other elements in the same relative order.	KL6
22	Given a singly linked list with N nodes and a number X. The task is to find the node at the given index (X)(1 based index) of linked list.	KL5
23	Given a Linked List and a number N, write a function that returns the value at the Nth node from the end of the Linked List.	KL5
24	Given a linked list, check if the linked list has a loop (cycle) or not. The below diagram shows a linked list with a loop. 	KL6
25	Given two sorted arrays arr1[] and arr2[] of sizes n and m in non-decreasing order. Merge them in sorted order without using any extra space. Modify arr1 so that it contains the first N elements and modify arr2 so that it contains the last M elements.	KL5
26	Given a string S of size N, the task is to sort the string without changing the position of vowels.	KL5
27	Given an array arr[] of size N and a number K, where K is smaller than the size of the array. Find the K'th smallest element in the given array. Given that all array elements are distinct.	KL5
28	Given the arrival and departure times of all trains that reach a railway station, the task is to find the minimum number of platforms required for the railway station so that no train waits. We are given two arrays that represent the arrival and departure times of trains that stop.	KL6
29	Given two strings 'str1' and 'str2' of size m and n respectively. The task is to remove/delete and insert the minimum number of characters from/in str1 to transform it into str2. It could be possible that the same character needs to be removed/deleted from one point of str1 and inserted at some another point.	KL6
30	Given are N ropes of different lengths, the task is to connect these ropes into one rope with minimum cost, such that the cost to connect two ropes is equal to the sum of their lengths.	KL6
31	Given a binary search tree, task is to find Kth largest element in the binary search tree.	KL6
32	Given a weighted graph and a source vertex in the graph, find the shortest paths from the source to all the other vertices in the given graph.	KL6

STUDENT-CENTERED LEARNING (Seff-Learning towards lifelong learning)

Self-Learning (it's a typical course-based project to be carried out by a whole class in groups of four students each; they should exhibit higher level BTLs)

The students, in a group, are expected to conceive an idea based on the content (objectives/outcomes) and apply the suitable knowledge to demonstrate their learning.

A) COURSE-BASED PROJECT (Psychomotor skills)

To enhance their skill set in the integrated course, the students are advised to execute course-based design projects. Some sample projects are given below:

S. No	Suggested Projects	KL
1	Comparing Kruskal and Prim's algorithm in MST.	KL5
2	Greedy and Backtracking Algorithm Comparison in Graph Coloring Problem.	KL5
3	Build the game of Capsa Susun using Greedy Algorithm	KL6
4	Solving Travelling Salesman Problem Using Greedy Algorithm and Brute Force Algorithm	KL3
5	Design and Analysis 0/1 Knapsack Problem.	KL6
6	Design Coin Change Problem using Brute Force and Greedy Algorithm	KL6
7	Comparing Exhaustive Search Algorithm and Greedy Algorithm in job Scheduling Problem	KL5
8	Build a Cash Flow Minimiser	KL6
9	Build a CB Mario game using Dynamic Programming Optimisation.	KL6
10	Build an application of Sudoku game using Backtracking method	KL6
11	Build a Snakes & Ladders game, challenge the player to win in minimum number of moves. You can use BFS to compute it.	KL6
12	Make an application like Google Maps - You can use Dijkshtra's algorithm to find the shortest paths, A* Search for more efficient & real time use.	KL6
13	Create a Binary Search Algorithm to Search for a Value with a Certain Precision	KL6

https://www.geeksforgeeks.org/learn-data-structures-and-algorithms-dsa-tutorial/?ref=shm

14	Create Space Efficient Algorithm for Subset Sum	KL6
15	Design the game- Place eight queens on an 8 × 8 chessboard so that no queen attacks another queen.	KL6
16	Build a game like SPACE- SHOOTER.	KL6
17	Make a PHONEBOOK using TRIE Data structure	KL6
18	Use a string compression algorithm, like run length encoding or Huffman coding	KL3
19	Build. Game like Flappy Bird.	KL6
20	Build a snake and Ladders game, challenge the player to win in minimum number of moves. You can use BFS to compute it.	KL6

B) SELF-LEARNING THROUGH MOOCs (Cognitive Skills):

1. Code tantra
2. https://www.coursera.org/courses?query=algorithm%20design
3.
4.
5.